



UNIVERSITÀ
DI TORINO

Dipartimento di Informatica

Tecnologie del Linguaggio Naturale

Parte II, Daniele Radicioni

Federico Chirco

Anno Accademico 2024/2025

Relazione Esercitazione 1.1

Concept Similarity

1. Introduzione

L'obiettivo di questa esercitazione è valutare la capacità di alcune **misure di similarità concettuale basate su WordNet** di approssimare i giudizi di similarità forniti da esseri umani.

La similarità semantica tra concetti è un problema centrale nel Natural Language Processing, con applicazioni che includono information retrieval, question answering e word sense disambiguation.

In questo lavoro viene utilizzato **WordNet**, una risorsa lessicale che organizza i significati delle parole in una gerarchia concettuale, permettendo di definire misure di similarità fondate sulla struttura tassonomica.

Le prestazioni delle misure implementate vengono valutate sul dataset **WordSim353**, che contiene coppie di parole annotate con un punteggio di similarità media assegnato da annotatori umani.

Le misure considerate sono:

- **Wu & Palmer**
- **Shortest Path**
- **Leacock & Chodorow**

La qualità dei risultati viene valutata tramite gli indici di correlazione di **Pearson** e **Spearman** rispetto ai punteggi umani.

2. Struttura del codice

Il codice è organizzato in una pipeline modulare composta dalle seguenti fasi principali.

2.1 Preparazione delle risorse

Viene inizialmente verificata la disponibilità di WordNet tramite NLTK.

Successivamente viene calcolato il valore di `depthMax`, definito come la **massima profondità della tassonomia nominale**, necessario per le misure basate sulla distanza.

Il valore è ottenuto iterando su tutti i synset dei nomi (`pos='n'`) e calcolando la massima profondità minima (`min_depth`) riscontrata.

Questo parametro è fondamentale per le misure basate sulla distanza (Shortest Path e Leacock & Chodorow), poiché consente di normalizzare i valori di similarità rispetto all'“altezza” della gerarchia, evitando l'uso di costanti arbitrarie.

2.2 Implementazione delle misure di similarità

2.2.1 Funzione `wu_palmer_score`

Questa funzione implementa la misura di **Wu & Palmer**, che calcola la similarità tra due sensi in funzione della **profondità** dei due termini e del loro antenato comune più vicino Lowest Common Hypernym (LCS).

La funzione recupera tutti gli iperonimi comuni ai due synset e seleziona quello con profondità massima.

Le profondità dei due sensi e del LCS vengono quindi utilizzate per calcolare il valore di similarità secondo la formula:

$$cs(s_1, s_2) = \frac{2 \cdot depth(LCS)}{depth(s_1) + depth(s_2)}$$

L'intuizione è che due concetti sono simili se condividono un antenato comune molto specifico e profondo e se loro stessi non sono troppo distanti da quell'antenato.

L'aggiunta di una unità alle profondità consente di evitare problemi legati alla presenza della radice della gerarchia a profondità zero.

2.2.2 Funzione `shortest_path_score`

Questa funzione implementa la misura di **Shortest Path**, che definisce la similarità come funzione inversa della **distanza** strutturale tra due sensi nella gerarchia di WordNet.

La distanza viene calcolata come il **numero di archi** nel cammino più breve tra i due synset (quanti passi separano i nodi). Il valore di similarità viene quindi ottenuto sottraendo tale distanza dal doppio della profondità massima della tassonomia, ottenendo un valore alto per parole vicine e basso per parole lontane:

$$sim_{path}(s_1, s_2) = 2 \cdot depthMax - len(s_1, s_2)$$

In assenza di un cammino valido tra i due synset, la funzione restituisce un valore nullo.

2.2.3 Funzione `leacock_chodorow_score`

Questa funzione implementa la misura di **Leacock & Chodorow**, simile allo Shortest Path, ma introduce una normalizzazione logaritmica della distanza tra i sensi.

Partendo dalla lunghezza del cammino minimo tra i synset, la funzione calcola la similarità come:

$$sim_{LC}(s_1, s_2) = -\log \frac{len(s_1, s_2)}{2 \cdot depthMax}$$

L'aggiunta di una unità alla distanza consente di evitare il caso di logaritmo di zero quando i due sensi coincidono.

Rispetto alla misura di Shortest Path, il log rende la funzione più stabile e meno sensibile a variazioni piccole.

2.3 Similarità tra termini

Poiché le misure operano sui sensi e non direttamente sulle parole, la similarità tra due termini viene calcolata come il **massimo valore di similarità tra tutte le coppie di sensi nominali** dei due termini:

$$\text{sim}(w_1, w_2) = \max_{c_1 \in s(w_1), c_2 \in s(w_2)} [\text{sim}(c_1, c_2)]$$

L'ipotesi è che, messi insieme, i due termini si auto-disambiguino (si sceglie l'interpretazione che li rende più simili).

2.4 Elaborazione del dataset WordSim353

Il dataset WordSim353 viene caricato tramite la libreria pandas e processato iterativamente. Per ciascuna coppia di parole presenti nel dataset, il codice calcola i punteggi di similarità utilizzando le tre misure implementate e memorizza i risultati in nuove colonne del DataFrame.

Questa fase consente di ottenere, per ogni coppia di termini, sia il punteggio umano di riferimento sia i punteggi prodotti automaticamente dal sistema.

2.5 Valutazione dei risultati

Nell'ultima fase vengono calcolati gli indici di correlazione di **Pearson** e **Spearman** tra i punteggi di similarità prodotti dal sistema e i punteggi umani presenti nel dataset.

- L'**indice di Pearson** valuta la correlazione **lineare** tra i valori numerici. Indica quanto bene i punteggi crescono in modo proporzionale a quelli umani.
- L'**indice di Spearman** misura quanto l'algoritmo produce lo **stesso ordinamento** delle coppie rispetto agli umani (ranking).

L'uso congiunto di entrambe le metriche consente una valutazione più completa delle prestazioni delle misure di similarità implementate.

3. Analisi dei Risultati

3.1 Indici di Pearson e Spearman

I risultati ottenuti sono riassunti nella Tabella seguente:

Misura	Pearson	Spearman
Wu & Palmer	0.27	0.32
Shortest Path	0.15	0.30
Leacock & Chodorow	0.31	0.30

I valori di correlazione indicano una **correlazione positiva ma moderata** tra le misure basate su WordNet e i giudizi umani, in linea con quanto riportato in letteratura per approcci simbolici.

- **Wu & Palmer** ottiene il miglior valore di **Spearman**, suggerendo una buona capacità di riprodurre l'ordinamento delle coppie di parole in termini di similarità.
- **Shortest Path** risulta la misura meno performante, in particolare secondo Pearson, confermando la forte sensibilità della distanza pura alla **densità non uniforme** della tassonomia di WordNet.
- **Leacock & Chodorow** ottiene il miglior valore di **Pearson**, suggerendo una migliore approssimazione dei valori numerici umani.

I risultati mostrano che le misure di similarità basate su WordNet presentano una correlazione moderata con i giudizi umani. Wu & Palmer risulta la migliore in termini di Spearman, indicando una buona capacità di ordinamento semantico, mentre Leacock & Chodorow ottiene la miglior correlazione di Pearson grazie alla normalizzazione logaritmica. Shortest Path risulta la meno affidabile, confermando la sensibilità della distanza pura alla densità non uniforme della tassonomia di WordNet.

3.2 Misure di similarità

L'ispezione delle prime righe del dataset evidenzia che **coppie tassonomicamente molto vicine** (es. *tiger-cat*) **ricevono punteggi elevati**, coerenti con i giudizi umani. Tuttavia, coppie fortemente associate dal punto di vista funzionale o pragmatico, ma non gerarchicamente correlate (es. *computer-keyboard*) mostrano discrepanze tra rispetto alle valutazioni umane.

	Word 1	Word 2	Human (mean)	Wu_Palmer	Shortest_Path	Leacock_Chodorow
0	love	sex	6.77	0.923077	35	2.890372
1	tiger	cat	7.35	0.965517	35	2.890372
2	tiger	tiger	10.00	1.000000	36	3.583519
3	book	paper	7.46	0.875000	34	2.484907
4	computer	keyboard	7.62	0.823529	33	2.197225

Il caso di *telephone-communication* (non visibile nella figura) è ancora più indicativo: nonostante l'elevato punteggio umano (7.50), la misura Wu & Palmer restituisce un valore molto basso (0.17). Questo risultato è spiegabile dal fatto che *communication* è un concetto astratto, mentre *telephone* è un artefatto concreto; i due termini condividono soltanto antenati molto generici nella gerarchia, rendendo il Lowest Common Subsumer poco informativo dal punto di vista semantico.

4. Conclusioni

L'esperimento conferma che le misure di similarità basate su WordNet costituiscono un solido baseline simbolico per il confronto semantico, ma non sono sufficienti a replicare completamente il giudizio umano. In particolare:

- risultano efficaci per concetti collegati da relazioni *is-a*
- falliscono nel catturare relazioni associative non tassonomiche

Nonostante questi limiti, le misure basate su WordNet rappresentano una baseline solida e interpretabile, utile per comprendere i fondamenti della similarità semantica e per confronti con approcci distribuzionali più moderni.

Relazione Esercitazione 1.2

Word Sense Disambiguation con SemCor

1. Introduzione

La **Word Sense Disambiguation (WSD)** è il compito di determinare quale senso di una parola polisemica è attivato in un determinato contesto. Si tratta di un problema centrale nel Natural Language Processing, in quanto molte applicazioni linguistiche richiedono una rappresentazione semantica non ambigua delle parole.

In questa esercitazione viene implementato un approccio **knowledge-based** alla WSD basato sull'algoritmo di **Simplified Lesk**, che sfrutta le definizioni (gloss) e gli esempi contenuti in **WordNet**. La valutazione del sistema viene condotta utilizzando **SemCor**, un corpus annotato manualmente con i sensi WordNet corretti, che rappresenta un gold standard ampiamente utilizzato in letteratura.

L'obiettivo dell'esperimento è duplice:

1. valutare l'accuratezza del sistema su un sottoinsieme casuale di frasi di SemCor;
 2. stimare la robustezza delle prestazioni tramite ripetute esecuzioni randomizzate.
-

2. Struttura del codice

Il codice è suddiviso in quattro blocchi principali: pre-processing, disambiguazione (Lesk), estrazione dei dati da SemCor ed esecuzione sperimentale con valutazione.

2.1 Pre-processing

Costanti globali: stopword e lemmatizzatore

Il codice inizializza due risorse globali:

- un insieme di **stopword** inglesi, usato per rimuovere parole funzionali poco informative (es. *the, of, and*),
- un **lemmatizzatore** WordNet, usato per ridurre le varianti morfologiche alla forma base (lemma).

Questa scelta è motivata dal fatto che Lesk si basa sull'**overlap lessicale**: ridurre il rumore e normalizzare le forme incrementa la probabilità di sovrapposizione significativa tra contesto e definizioni.

Funzione `tokenize(text)`

Questa funzione tokenizza un testo (tipicamente definizioni ed esempi di WordNet) estraendo **solo sequenze alfabetiche** e convertendole in minuscolo. In tal modo:

- si eliminano punteggiatura e numeri,
- si produce un insieme di token "puliti" e confrontabili con quelli del contesto.

Funzione `normalize_words(words)`

Questa funzione applica la normalizzazione al livello di lista di parole:

- rimuove le stopword,
 - lemmatizza ogni token rimanente.
-

2.2 Implementazione dell'algoritmo di Simplified Lesk

Funzione `simplified_lesk(word, sentence_tokens, pos="n")`

Questa funzione implementa l'algoritmo di Simplified Lesk per disambiguare una parola target all'interno di una frase.

- **Normalizzazione della parola target:** la parola viene convertita in minuscolo e gli spazi vengono sostituiti con underscore (_). Ciò permette di gestire correttamente espressioni multiword nel formato atteso da WordNet.
- **Costruzione del contesto:** dal testo della frase (`sentence_tokens`) viene costruito un insieme (set) di parole normalizzate (stopword rimosse + lemmatizzazione).
- **Recupero dei sensi:** vengono recuperati tutti i synset della parola target in WordNet limitandosi ai **sostantivi** (`pos="n"`).
- **Scelta iniziale del best sense:** come baseline, viene impostato il primo synset come candidato migliore (`synsets[0]`), in quanto spesso rappresenta il “most frequent sense” secondo l'ordinamento di WordNet.
- **Costruzione della signature e calcolo overlap:** per ciascun senso, viene costruita una **signature** composta da:
 - parole della definizione (**gloss**),
 - parole degli **esempi**,
 - nomi dei **lemma** associati al synset (sinonimi).

Signature e contesto vengono normalizzati nello stesso modo e l'overlap viene calcolato come cardinalità dell'intersezione tra insiemi (`signature n context`).

Il contesto utilizzato dall'algoritmo è rappresentato mediante un modello di tipo **bag-of-words**, in cui le parole della frase vengono considerate come un insieme non ordinato di token normalizzati.

- **Decisione finale:** il senso con overlap massimo viene selezionato come predizione.

La funzione restituisce:

- il synset predetto,
 - il valore di overlap massimo (utile per interpretabilità e debug).
-

2.3 Estrazione delle frasi e dei candidati da SemCor

SemCor contiene frasi annotate in cui alcune parole sono legate al loro senso WordNet corretto. Tuttavia, la rappresentazione delle frasi in NLTK non è una semplice lista di stringhe, ma una struttura mista che può includere alberi (Tree) dove ogni parola è impacchettata con metadati (POS tag e Synset).

Funzione `flatten_sentence(sent)`

Questa funzione ha il compito di:

1. **ricostruire la frase come sequenza lineare di token** (lista di stringhe),
2. **estrarre i candidati disambiguabili**, ovvero i sostantivi annotati semanticamente, insieme al loro synset gold.

Per ciascun elemento della frase (chunk):

- se chunk è un Tree, rappresenta un'annotazione semantica; si estraggono le foglie (parole), si aggiungono ai token della frase, e si recupera l'etichetta semantica (gold sense) convertendola in synset tramite `synset()`;
- dal sotto-albero interno (quando presente) si recupera anche il POS tag, e si selezionano come candidati solo i token con **POS nominale** (prefisso "NN");
- se chunk non è un Tree, viene trattato come parte non annotata (stringa o lista) e viene comunque aggiunto ai token della frase.

Output:

- `tokens`: frase lineare utilizzabile come contesto per Lesk,
- `candidates`: lista di coppie (`surface`, `gold_synset`).

2.4 Esecuzione sperimentale e calcolo dell'accuratezza

Funzione `one_run(n_sent=50, seed=None, verbose=False, show_n=10)`

Questa funzione esegue una singola run sperimentale:

- **selezione random di 50 frasi**: vengono estratte casualmente `n_sent=50` frasi da SemCor mediante campionamento senza ripetizione (`rng.sample`).
- **selezione random del target**: per ciascuna frase viene estratto l'insieme dei candidati (sostantivi annotati); se presenti, ne viene scelto uno a caso (`rng.choice`).
- **predizione del senso**: viene applicato `simplified_lesk` sul target e sul contesto della frase.
- **valutazione per istanza**: la predizione viene confrontata con il synset gold della parola target. Ogni predizione contribuisce al conteggio `correct/total`.
- **debug controllato**: se `verbose=True`, la funzione stampa i dettagli delle prime `show_n` istanze (frase, target, overlap, gold sense, predicted sense e correttezza), permettendo un'analisi qualitativa.

La run restituisce l'**accuracy**, definita come:

$$accuracy = \frac{\# \text{ predizioni corrette }}{\# \text{ predizioni totali }}$$

3. Risultati

L'esperimento è stato eseguito su **10 run indipendenti**, ciascuna basata sulla selezione casuale di **50 frasi** da SemCor e sulla disambiguazione di **un sostantivo per frase**.

I risultati ottenuti sono i seguenti:

- **Accuratezza media: 69.4%**
- **Deviazione standard (popolazione): 0.080**

Le singole accuracy per run variano approssimativamente tra **il 56% e l'83%**, riflettendo la dipendenza dell'algoritmo dalla specifica selezione delle frasi e dei termini target.

Nel complesso, le prestazioni risultano **nettamente superiori al caso**, confermando che l'algoritmo di Simplified Lesk è in grado di sfruttare efficacemente le informazioni lessicali fornite da WordNet.

3.1 Analisi qualitativa degli esempi

L'analisi degli output di debug permette di comprendere meglio il comportamento del sistema.

Casi corretti

In molti esempi, il sistema seleziona correttamente il senso grazie a un overlap anche minimo ma semanticamente rilevante. Termini come *office*, *bed*, *morality* e *crowd* vengono spesso disambiguati correttamente perché:

- il contesto è coerente con un senso dominante,
- i gloss di WordNet contengono parole sufficientemente distintive.

In questi casi, anche un overlap pari a 1 è sufficiente a guidare correttamente la decisione.

Casi errati

Gli errori osservati mettono in luce i limiti strutturali dell'approccio:

- **pitch**: in un contesto sportivo, il sistema seleziona il senso acustico invece di quello legato al baseball. Questo errore è dovuto alla mancanza di parole chiave sportive nel gloss e all'assenza di una rappresentazione tematica globale del contesto.
- **home**: il sistema confonde sensi concettualmente molto vicini (*abitazione* vs *luogo di origine*), evidenziando la difficoltà di Lesk nel distinguere polisemia fine.
- **stall**: il contesto suggerisce un ambiente animale, ma il gold sense annotato in SemCor è più generico (*booth*). L'overlap lessicale non è sufficiente a discriminare tra sensi spaziali semanticamente affini.

Questi esempi mostrano come Simplified Lesk tenda a funzionare bene quando i sensi sono ben separati, ma incontra difficoltà quando le definizioni condividono vocabolario o quando il contesto richiede inferenze pragmatiche.

4. Conclusioni

I risultati mostrano che:

- l'algoritmo raggiunge un'accuratezza media di circa **69%**, in linea con i valori riportati in letteratura per approcci knowledge-based non supervisionati;
- le prestazioni sono sensibili alla selezione casuale delle frasi e dei target, come evidenziato dalla deviazione standard osservata;
- l'overlap lessicale costituisce un criterio efficace ma limitato per la disambiguazione semantica.

I principali limiti dell'approccio riguardano:

- la dipendenza da gloss brevi e spesso poco informativi;
- l'assenza di pesatura delle parole (tutte contribuiscono allo stesso modo all'overlap);
- la difficoltà nel trattare sensi semanticamente molto vicini.

Nonostante tali limiti, Simplified Lesk rappresenta una **baseline solida, interpretabile e concettualmente semplice**, utile per comprendere i fondamenti della WSD e per confronti con approcci più avanzati, come il **Corpus Lesk** o i metodi supervisionati e distribuzionali.

Relazione Esercitazione 2

Modeling social media and literary language

1. Introduzione

Lo scopo dell'esercitazione è costruire e confrontare **language model statistici a n-grammi** (bi-gram e tri-gram). Un modello a n-grammi stima la probabilità della prossima parola in funzione delle ultime $n - 1$ parole generate:

- **Bigram:** $P(w_i | w_{i-1})$
- **Trigram:** $P(w_i | w_{i-2}, w_{i-1})$

L'esercitazione è divisa in due parti:

- A. **Tweet:** addestrare modelli bigram e trigram su due collezioni distinte (Barack Obama vs Cristiano Ronaldo), generare tweet e confrontare lo stile.
 - B. **Moby-Dick:** addestrare modelli bigram e trigram sul testo del romanzo e generare brevi brani, osservando la differenza qualitativa tra i due ordini.
-

2. Struttura del codice

Il progetto è organizzato in 3 file principali:

- `lm_utils.py`: funzioni riutilizzabili (preprocessing, training, generazione, statistiche).
- `run_tweets.py`: esecuzione Parte A sui tweet.
- `moby_dick.py`: esecuzione Parte B su Moby-Dick.

2.1 `lm_utils.py`

Pulizia e tokenizzazione

La funzione `clean_tweet()` normalizza pattern tipici dei social:

- URL → `<URL>`
- mention → `<USER>`
- numeri → `<NUM>`

Questo riduce la sparseness (molte stringhe uniche come link e username) e rende lo stile più apprendibile dal modello.

La tokenizzazione è fatta con `word_tokenize`, che separa parole e punteggiatura.

Training: `train_lm(texts, n)`

La funzione:

1. Pulisce e tokenizza ogni testo.
2. Usa `padded_everygram_pipeline(n, tokenized)` per:
 - aggiungere **padding** (`<s>`, `</s>`) così il modello impara inizio/fine frase,

- generare gli **n-gram** necessari,
 - costruire il **vocabolario** dei token.
3. Crea un modello Laplace(n) (smoothing add-one) per evitare probabilità zero per n-gram non osservati.
 4. Chiama `fit(train_data, vocab)` per stimare conteggi e probabilità.

Generazione: `generate_from_lm(lm, max_tokens, seed)`

La generazione avviene in modo incrementale:

1. Inizializza il contesto con `<s>` ripetuto $n - 1$ volte (bigram: 1 token, trigram: 2 token).
2. Ad ogni passo:
 - calcola la distribuzione $P(\cdot | \text{contesto})$,
 - campiona una parola (`lm.generate(...)`),
 - aggiorna il contesto mantenendo gli ultimi $n - 1$ token (“sliding window”),
 - termina se esce `</s>` oppure se raggiunge `max_tokens`.

Statistiche di stile: `style_stats(texts)`

Calcola metriche utili per confrontare gli autori:

- numero di tweet
- lunghezza media in token
- percentuale tweet con URL/mention/hashtag
- percentuale di retweet (tweet che iniziano con “RT”)

3. Risultati

3.1 Parte A – Tweet (BarackObama vs Cristiano)

Dataset e setup

Dal file `tweets.csv` vengono estratti i tweet in lingua inglese (`language == "en"`) per i due autori:

- BarackObama
- Cristiano

Per ciascun autore vengono addestrati:

- **LM bigram (n=2)**
- **LM trigram (n=3)**

La generazione è limitata a ~25 token per tweet.

Statistiche di stile

Dai risultati ottenuti:

Autore	N (tweet)	Avg tokens	URL%	@%	#%	RT%
BarackObama	2857	21.26	0.797	0.101	0.571	0.0000
Cristiano	2134	20.58	0.695	0.383	0.251	0.0014

Interpretazione:

- Obama presenta un uso molto alto di **URL** e soprattutto di **hashtag**, coerente con uno stile “campagna/call-to-action” (iniziative, temi, slogan, link informativi).
- Cristiano usa molte più **mention** (@), tipiche di comunicazione social/brand/engagement e collaborazioni.

Esempi generati (qualitativo)

In generale:

- **Bigram**: testi più spezzati e con transizioni meno coerenti.
- **Trigram**: maggiore fluidità e sequenze più realistiche.

Obama / BIGRAM

- “LIVE : <URL> <URL> # GetCovered today : <URL>”
- “Today our immigration system ... # ItsOnUs”

Obama / TRIGRAM

- “Say you ’re ready to # ActOnClimate”
- “Despite Judge Garland a fair hearing ... <URL>”

Cristiano / BIGRAM

- “We have all the launch of # CR7SelfieApp <URL>”
- “First training. Thanks Sweden <URL>”

Cristiano / TRIGRAM

- “Check out my new gift from <USER>! ... Register now : <URL>”
- “Getting ready for this # celebrate15m ... <URL>”

Osservazione chiave: il trigram riproduce meglio pattern tipici:

- Obama: “invito all’azione + tema/hashtag + link”
- Cristiano: “promo/engagement + mention + link + emoji/hashtag brand”

3.2 Parte B – Moby-Dick

Dataset e setup

Il testo originale preso dal file moby-dick.txt viene segmentato in frasi con sent_tokenize, filtrando frasi troppo corte (<5 parole) in modo da eliminare rumore testuale.

Tramite il modulo `lm_utils`, il testo è stato ripulito da eccessi di spazi e i termini numerici sono stati normalizzati per ridurre la dispersione del vocabolario. Anche qui vengono addestrati bigrammi ($n=2$) e trigrammi ($n=3$). La generazione è limitata a ~60 token per brano.

Esempi generati e confronto bigram vs trigram

Bigram (più “rumoroso”): frasi meno coerenti, salti improvvisi, punteggiatura e costruzioni talvolta irregolari.

- His whole visible from the casks had dragged down, though seated on one whole, Queequeg, like his senses.
- The Parsee occupied ship biscuit were mostly all, furiously at least chance ; and therefore, link with cords woven by a whole, my fond, two pledges that they have not as a mower a spire on high social quarrel as standing in almost solid rib only foamin ’ t our pains ye see a true enough

Trigram (più coerente): frasi più lunghe e strutturate, registro più vicino alla prosa narrativa ottocentesca, con frammenti dialogici plausibili (es. “roared the captain...”).

- But as all Asia, or shun the place, he steadily depresses the butt-end in his boat ; yet, it would be to replenish his reservoir of air, especially upon the school of females, you had best cut away from each other ’ s eyes had once been a very curious to watch that lord.
- Jumped from a creature identical with the corpses and ghosts of these whale cemeteries, and every hour passed by ruthless hands, radiates without end from God ; Himself!

Il trigram mostra effettivamente una coerenza locale più alta rispetto al bigram, pur restando un modello a memoria corta (dipendenza solo dalle ultime due parole).

4. Conclusioni

L’esperimento svolto su due domini testuali diversi (tweet e prosa letteraria) mostra in modo chiaro l’impatto dell’ordine del modello n -gram e dei vincoli di questo approccio.

Aumentare l’ordine del modello (da **bigram** a **trigram**) migliora la **coerenza locale** del testo generato: il trigram, usando due token di contesto, produce sequenze più plausibili rispetto al bigram, che risulta più frammentato. Questo effetto è evidente sia nei tweet sia nei brani generati da *Moby-Dick*, dove il trigram restituisce frasi più strutturate e più vicine al registro narrativo.

Il principale limite dell’approccio n -gram resta la **sparseness**, soprattutto per trigram: molte combinazioni non compaiono nel corpus. Lo **smoothing di Laplace** evita probabilità nulle, ma può rendere la distribuzione più “piatta” e talvolta meno naturale. Infine, la normalizzazione di URL/mention/numeri è risultata importante per ridurre rumore e variabilità nei tweet.